

ÖFFENTLICHES TECHNISCHES WHITEPAPER

ODRE PQC v0.2.9

Application-Embedded Post-Quantum Security Boundary

Sicherheitsentwicklung · Bedrohungsprüfung · Plattformübergreifende Evidenz

DEUTSCHE AUSGABE

September 2026

Inhalt

01 **Entwicklungs- und Sicherheitsfortschritt**
v0.1.1 bis v0.2.9

02 **Bedrohungsmodell und Prüfung**
Replay, Sequence und Native Identity

03 **Plattformspezifische Verifikation**
Windows Server 2022 und Ubuntu 24.04 LTS

04 **Artefakte und Evidenz**
Versiegeltes Wheel, Runtime und Berichte

05 **Fazit**
Beobachtete Entwicklung und finales Artefakt

Produktübersicht und Architektur werden in einem separaten Dokument behandelt. Dieses Whitepaper konzentriert sich auf Sicherheitsentwicklung und ausführbare Evidenz.

Öffentliches Technisches Whitepaper

1. Entwicklungs- und Sicherheitsfortschritt

ODRE PQC wurde von v0.1.1 bis v0.2.9 als anwendungsgebundene Post-Quanten-Sicherheitsgrenze weiterentwickelt. Die Entwicklung folgte keinem bloßen Funktionszuwachs. Jede Stufe nahm beobachtete Fehlerbilder auf, änderte die dafür verantwortliche Struktur und prüfte die Änderung erneut auf den unterstützten Plattformen. Die hier genannten Ergebnisse beziehen sich jeweils auf das bezeichnete Artefakt, die Testumgebung und den ausgeführten Umfang.

1.1 v0.1.1 – Ausgangspunkt der prüfbaren Linie

v0.1.1 etablierte die grundlegende Trennung zwischen öffentlichem und geschütztem Pfad. Admission, Handshake, Session, Sequenzprüfung und Handler-Aufruf wurden als unterscheidbare Zustände behandelt. Dadurch konnten spätere Prüfungen nicht nur einen HTTP-Erfolg, sondern die Reihenfolge und Einmaligkeit sicherheitsrelevanter Übergänge beobachten.

1.2 v0.1.2 – Clean-Room-Prüfung und drei konkrete Befunde

In der Clean-Room-Prüfung wurden zunächst 17 von 20 Fällen erfüllt. Drei Abweichungen betrafen nicht die Auswahl der PQC-Verfahren, sondern die Verbindlichkeit der Laufzeitlogik: unvollständige Bindung eines Anfragekontexts, ein uneindeutiger Fehlerpfad und eine Zustandsbereinigung, die bei einem Abbruch nicht streng genug war. Die Ursache lag in Übergängen, die funktional erlaubt, sicherheitlich aber nicht eindeutig waren.

Die Korrektur verschob die Entscheidung in explizite, fail-closed geprüfte Zustände. Ein Handler durfte erst nach erfolgreicher Prüfung aller vorgelagerten Bedingungen laufen. Abbruchpfade löschten den betroffenen Zwischenzustand. Die Wiederholungsprüfung erreichte anschließend 20 von 20 Fällen.

1.3 v0.1.3 – Konfiguration und Paketintegrität

v0.1.3 ergänzte die Trennung von Produktlogik und Konfigurationsprüfung. Sechs von sechs Konfigurationsfällen wurden ausgeführt. Das Python-Wheel wurde zusätzlich über seinen RECORD-Inhalt geprüft; 31 von 31 Einträgen waren vorhanden und konsistent. Damit wurde die Aussage über den getesteten Code an ein konkret identifizierbares Paket gebunden.

1.4 v0.1.4 – Statusmodell und Sicherheitsregression

Die Statusdarstellung wurde so überarbeitet, dass Diagnoseinformationen keine Schutzentscheidung ersetzen. Fünfzehn Statusfälle und vier ergänzende Zustandsfälle wurden geprüft. Die Sicherheitsregression umfasste 41 von 41 bestandene Fälle, während RECORD auf 32 von 32 Einträge anwuchs. Entscheidend war die Trennung zwischen beobachtbarem Status und autorisierender Entscheidung.

1.5 v0.2.4 bis v0.2.5 – Plattformgrenze und Ubuntu-ABI

Beim Ausbau des nativen Laufzeitpfads zeigte sich ein Ubuntu-spezifischer ABI-Fehler. Die Schnittstelle zwischen Python-Paket, Loader und nativer Bibliothek war auf einer Plattform funktionsfähig, aber nicht ausreichend plattformneutral beschrieben. Die Korrektur führte eine explizite Platform-Adapter-Grenze ein und machte Dateiname, Export-Symbole, Manifest und erwartete Hashes zu prüfbareren Eingaben.

Die nachfolgende Prüfung bestätigte, dass Windows-DLL und Ubuntu-SO denselben übergeordneten Sicherheitsvertrag erfüllen, obwohl Ladevorgang und Dateisystemsemantik unterschiedlich sind.

1.6 v0.2.6 – Native Trust, TOCTOU und FD Identity

v0.2.6 konzentrierte sich auf die Frage, ob die geprüfte Datei tatsächlich das geladene Objekt ist. Ein reiner Pfad- oder Hashvergleich vor dem Laden lässt ein Zeitfenster zwischen Prüfung und Nutzung. Deshalb wurden Dateideskriptor-, Inode- und Objektidentität in die Vertrauenskette aufgenommen. Symlink-, Junction- und Reparse-Fälle wurden getrennt von normalen Dateien behandelt.

Das geprüfte Wheel dieser Stufe hatte den SHA-256-Wert

[4756FC9B219DAB4E2BF9C90FA066CFFD0D5DEA94FEF03DCE0B18DED55E5A7C7A](#). Dieser Wert bezeichnet ausschließlich das damalige Prüfartefakt und ist nicht mit dem späteren finalen Wheel zu verwechseln.

1.7 v0.2.8 bis v0.2.9 – Race-Härtung und finales Sealing

Der Befund F-028-01 führte zu einer erneuten Prüfung der Beziehung zwischen Pfad, geöffnetem Objekt und geladener Bibliothek. In 600 Versuchen wurden insgesamt 1.212.203 Symlink-Wechsel erzeugt. 278 Läufe endeten mit erfolgreicher, identitätskonsistenter Nutzung; 322 Läufe brachen sicher ab. Die Zähler für einen kompromittierten Load, einen Handler-Aufruf nach fehlgeschlagener Prüfung und eine unerkannte Identitätsabweichung blieben jeweils bei null.

Die abschließende Änderung vereinigte Runtime-Manifest, Archivprüfung, Loaded-object Identity, Session-Lebenszyklus und Paketversiegelung in einer reproduzierbaren Prüfkette. Nach der Korrektur wurden die bereits berührten Race-, Lifecycle- und Cross-Platform-Fälle erneut ausgeführt, statt nur neue Tests hinzuzufügen.

2. Bedrohungsmodell und Sicherheitsprüfung

2.1 Replay und Sequence Race

Ein Angreifer kann gültige Chiffretexte erneut senden, Sequenzwerte parallel verwenden oder Antworten aus einer anderen Session einspielen. Die Prüfung beobachtete deshalb nicht nur Ablehnungsstatus, sondern auch, ob der geschützte Handler höchstens einmal ausgeführt wurde. Unter Windows wurden 1.280 konkurrierende Replay-Versuche, unter Ubuntu 2.880 Versuche ausgeführt. Doppelte Handler-Ausführungen: 0 auf beiden Plattformen.

2.2 Anfrage-, Antwort- und Kontextbindung

Die kryptografische Gültigkeit allein reicht nicht, wenn Methode, Pfad, Session oder Sequenz ausgetauscht werden können. ODRE PQC bindet den geschützten Inhalt an den verifizierten Anfragekontext. Prüfungen variierten Methode, Route, Session, Sequenz und Payload einzeln sowie in Kombination. Eine Antwort wurde an die zugehörige Anfrage und Session gebunden, sodass eine gültige Antwort nicht als Antwort auf einen anderen Kontext akzeptiert werden durfte.

2.3 Sitzungs- und Pfadübergreifende Angriffe

Cross-Session- und Cross-Path-Fälle prüften, ob Material aus einer legitimen Verbindung in einer anderen Verbindung oder Route wiederverwendet werden kann. Die erwartete Eigenschaft war eine Ablehnung vor dem Handler. Der relevante Beobachtungspunkt war deshalb sowohl der Protokollstatus als auch der Handler-Zähler.

2.4 Malformed, Oversized und HTTP-Smuggling-Formen

Fehlerhafte Längen, unbekannte Felder, abgeschnittene Frames, Mehrdeutigkeiten bei Headern und übergroße Eingaben wurden getrennt geprüft. Ziel war ein deterministischer Abbruch mit begrenztem Ressourcenverbrauch. HTTP-Smuggling-ähnliche Formen wurden an der Grenze zwischen Webserver-Parsen und geschützter Nachrichteninterpretation betrachtet; mehrdeutige Darstellungen durften nicht zu zwei verschiedenen Sicherheitsentscheidungen führen.

2.5 Fuzz, Soak und Load

Fuzzing variierte Struktur, Länge und Feldkombinationen. Soak- und Lasttests beobachteten über längere Laufzeiten Speicher, Zustand, Sequenzfortschritt und Fehlerbereinigung. Ein Test-Harness-Fehler wurde von einem Produktfehler getrennt: Ein Harness-Befund zählt nicht automatisch als kryptografischer oder protokollarischer Defekt, muss aber korrigiert werden, bevor sein Ergebnis als Evidenz verwendet wird.

2.6 TOCTOU, Symlink, Junction und Loaded-object Identity

Die native Vertrauensprüfung nahm einen lokalen Angreifer an, der Pfade austauschen, Links umbiegen oder Dateien zwischen Prüfung und Laden ersetzen kann. Unter Unix wurden FD- und Inode-Eigenschaften beobachtet; unter Windows wurden Reparse- und Loaded-module-Eigenschaften berücksichtigt. Der Kern der Prüfung war die Identität des tatsächlich geladenen Objekts, nicht nur der Name der Datei.

2.7 Provider-, Dependency- und Config-Injection

Manipulierte Provider-Pfade, unerwartete Abhängigkeiten und Konfigurationsüberschreibungen wurden als Eingaben der Trust Boundary behandelt. Die Runtime durfte nicht stillschweigend auf ein ungeprüftes Objekt zurückfallen. Fehlende oder abweichende Manifest-, Hash- oder Provider-Angaben führten zum Abbruch des geschützten Pfads.

2.8 Worker, Crash, Restart und Stale State

Concurrent Init und Worker Collision prüften, ob mehrere Prozesse denselben Initialisierungszustand widersprüchlich beanspruchen. Crash-, Restart- und Shutdown-Fälle beobachteten die Bereinigung von Session, Lease und Status. Alte Zustände durften nach einem Neustart nicht als neue Autorisierung wiederverwendet werden.

3. Plattformspezifische Verifikation

3.1 Windows Server 2022

Die Windows-Prüfung verwendete Windows Server 2022 AMD64 mit CPython 3.12.10. Sie umfasste Wheel-Installation, native DLL-Identität, OpenSSL-Laufzeit, PQC-Self-Tests, Replay-Races, Lifecycle und Paketintegrität. Die Windows-Runtime wurde durch SHA-256

[826303E72A76B10041D26113F465ADD8960825C71F50722D6792174E73F9D12C](#) identifiziert.

3.2 Ubuntu 24.04 LTS

Die Ubuntu-Prüfung verwendete Ubuntu 24.04 LTS AMD64 mit CPython 3.12.3. Sie wiederholte die Kernfälle mit der nativen SO und Unix-Dateisystemsemantik. Die Ubuntu-Runtime wurde durch SHA-256

[EFAD16698203371D07F25B429F6731B9C0954EEBF5B58A0024789EF0611A78D1](#) identifiziert.

3.3 Reale PQC-Runtime und Hybrid Handshake

Auf beiden Plattformen wurde OpenSSL 3.5.8 als tatsächlich geladene Laufzeit beobachtet. ML-KEM-768 und ML-DSA-65 wurden durch Self-Tests ausgeführt. Der Hybrid-Handshake kombinierte die vorgesehenen Rollen, ohne bei fehlender PQC-Verfügbarkeit unbemerkt auf einen schwächeren Schutzpfad zurückzufallen.

3.4 Dasselbe Wheel auf zwei Plattformen

Die plattformübergreifende Aussage bezieht sich auf dasselbe versiegelte Python-Wheel, gekoppelt an je eine plattformspezifische Native Runtime. Damit wird Python-Produktlogik gemeinsam geprüft, während DLL und SO über eigene Manifest- und Hashwerte identifiziert bleiben.

4. Artefakte und Evidenz

4.1 Finales Wheel

Das finale v0.2.9-Wheel wird durch SHA-256 [A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697](#) bezeichnet. Die RECORD-Prüfung bestätigte 45 von 45 Einträgen. Beginn und Ende der abschließenden Prüfung wurden gegen dieselbe Artefaktidentität verglichen.

4.2 Evidenzstruktur

Jeder belastbare Testdatensatz benötigt mindestens Test-ID, Plattform, Runtime-Version, Artefakthash, Konfiguration, Eingabe, erwartete Eigenschaft und beobachtetes Ergebnis. Berichte ohne diese Zuordnung sind erläuternd, ersetzen aber keine reproduzierbare Evidenz.

4.3 Reproduktion

Eine Reproduktion beginnt in einer sauberen Umgebung mit dem bezeichneten Wheel. Danach werden Runtime-Archiv und Manifest geprüft, die plattformspezifische Native Runtime geladen, [doctor](#) und [verify](#) ausgeführt und die Testfälle mit dokumentierter Parallelität wiederholt. Abschließend werden Wheel, RECORD, Runtime-Hash und Ergebnisdateien erneut gehasht.

5. Fazit

Von v0.1.1 bis v0.2.9 verlagerte sich die Sicherheit von impliziten Funktionspfaden zu expliziten, messbaren Zuständen: Handler-Ausführung nach vollständiger Prüfung, strenge Session- und Sequenzbindung, identitätsgebundene Native Runtime, bereinigter Lifecycle und versiegelte Artefakte. Die Windows- und Ubuntu-Prüfungen bestätigten den vorgesehenen Sicherheitsvertrag für die bezeichneten Kombinationen.

Die öffentlich nachprüfbaren Kennwerte sind das finale Wheel mit SHA-256

[A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697](#), RECORD 45/45, die beiden Runtime-Hashes sowie die angegebenen Race- und Replay-Zahlen. Diese Angaben ermöglichen die Zuordnung der Aussagen zu konkreten Artefakten und Beobachtungen.